Input : maybe this is why
Prediction : neutral
Confidence : 0.70
- - - -
Input : Let's start again
Prediction : neutral
Confidence : 0.83
- - - -
Input : This is the worst thing
Prediction : negative
Confidence : 0.93.

⇒ Fine-Tuning for Classification

Here, you'll learn how to fine-tune the Classify endpoint model on custom datasets, enhancing its performance on specific tasks.

↳ In the previous chapter, you used few-shot classification for a sentiment analysis task & got decent results with a small dataset.

↳ Let's say you actually have a decent amount of training dataset at your disposal. Then, you can leverage these additional data points to build a potentially better-performing classifier through a process known as fine-tuning.

↳ Fine-tuning is a powerful concept, and here's the reason, with fine-tuning, you can actually alter the model itself and customize it according to your task.

This means that what you get is a custom model that is fine-tuned to excel at a specific task & potentially can outperform the few-shot approach from prev chapter.

↳ Here, you'll learn how to fine-tune a model for classification.

↳ Step-by-Step Guide

To setup, we first import several tools.

```
import os
import Json
import numpy as np
import pandas as pd
from cohera import classifyExample
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score.
```

↳ We also import the Cohere module & create a client.

```
import cohere
co = cohere.Client("Cohere Api-key")
```

↳ Step ①: Prepare the Dataset

we'll use the Airline Travel Info^n System Intent classificat^n dataset. For demonstration purposes, we'll take just a small portion of the dataset: 1000 datapoints in total.

```
# Load the dataset to a dataframe
df = pd.read_csv('https://raw.githubusercontent.
                  com/cohere-ai/notebooks/
                  main/notebooks/data/
                  atis_subset.csv',
                  names = ['query', 'Intent'])
```

↳ The 1st thing we need is to create a training dataset, to be used for beilding the classifier, and a test dataset, to be used for testing the classifier performance. We will use 800 & 200 data points for these datasets, respectively.

```
df_train, df_test = train_test_split(df, test_size=200,
                                     random_state=21)
```

↳ Our goal is to train the classifier so it can predict the class of a new customer inquiry out of 8 classes as follows:

['atis_flight', 'atis_airfare', 'atis_ground_service', 'atis_flight_time', 'atis_airline', 'atis_quantity', 'atis_abbreviation', 'atis_aircraft']

↳ We transfor
style expected
def crea
for
if not
else

↳ The JSON
has "text" &
{"text":

↳ We transform the data to JSONL format to match the style expected by the classification endpoint

```
def create_classification_data (text, label):
    formatted_data = {
                "text" : text,
                "label" : label
            }
    return formatted_data

if not os.path.isfile ("data.jsonl"):
    print ("Creating jsonl file...")
    with open ("data.jsonl", 'w+') as file:
        for row in df_train.itertuples():
            formatted_data = create_classification_data
                                (row.query, row.intent)
            file.write (Json.dumps (formatted_data)
                                + '\n')

        file.close()
        print("Done")

else
    print ("data.jsonl file already exists")
```

↳ The JSONL file has a row for each example. Each example has "text" & "label" fields.

{"text": "what flights go from newyork to boston after 5 pm", "label": "atis_flight"}

↳ Step ② : Fine-Tune the Model

We kick off a fine-tuning job by navigating to the fine-tuning tab of open dashboard. Under "Classify", click on "Create a Classify model".

Next upload the .Jsonl file you just created as the training set by clicking on the "Training set" button. When ready, click on "Review data" to proceed the next step.

Then you'll see a preview of how the model will ingest your data. If anything is wrong with the data, the page will also provide suggested changes to fix the training file. Otherwise, if everything looks good, you can proceed to next step.

Finally, you'll see a page where you'll provide a nickname to your model. We used atis-classify-ft as the nickname for our model. Under "BASE MODEL", ensure "english" is selected.

Then click on "start training" to kick off the fine-tuning process. This will navigate you to a page where you can monitor the status of the model. A model that has finished fine-tuning will show the status as "READY".

↳ Step ③

Once the
to evaluate
tab of th
model ID
co.clas

↳ We fi
predicti

def cl

cre
BATC
y_
for

↳ Next, we
accuracy
f1 = f
-pure

Step ③ Use/Evaluate the Fine-Tuned Model

Once the model has finished fine-tuning, it's time to evaluate its performance. Navigate to the API tab of the fine-tuned model. There, you'll see the model ID that you should use when calling co.classify( ).

↳ We fill in the the model ID to generate test predictions

```
def classify_text_finetune(texts):
    classifications = co.classify(
        model = 'b2c94ac3-79 ----',
        inputs = texts,
        examples = None
    ).classifications.
    return [c.predictions[0] for c in classifications]

# create batches of texts & classify them.
BATCH_SIZE = 90 # The API accepts a max^m of 96 inputs
y_pred = [ ]
for i in range(0, len(df_test), BATCH_SIZE):
    batch_texts = df_test["query"][i: i+BATCH_SIZE].
                                    tolist()

    y_pred.extend(classify_text_finetune(batch_texts))
```

↳ Next, we calculate the model's test accuracy & F1 score

```
accuracy = accuracy_score(df_test["intent"], y_pred)
f1 = f1_score(df_test["intent"], y_pred, average='weighted')
print(accuracy, f1).
```

↳ Step③ Use/Evaluate the Fine-Tuned Model

Once the model has finished fine-tuning, it's time to evaluate its performance. Navigate to the API tab of the fine-tuned model. There, you'll see the model ID that you should use when calling co.classify( ).

↳ We fill in the the model ID to generate test predictions.

```
def classify_text_finetune(texts):
    classifications = co.classify(
        model='b2c94ac379 - - - ',
        inputs = texts,
        examples = None
    ).classifications.
    return [c.predictions[0] for c in classifications]

# create batches of texts & classify them.
BATCH_SIZE = 90 # The API accepts a max^m of 96 inputs
y_pred = []
for i in range(0, len(df_test), BATCH_SIZE):
    batch_texts = df_test["query"][i:i+BATCH_SIZE].
                                    tolist()

    y_pred.extend(classify_text_finetune(batch_texts))
```

↳ Next, we calculate the model's test accuracy & F1 score

```
accuracy = accuracy_score(df_test["intent"], y_pred)
f1 = f1_score(df_test["intent"], y_pred, average='weighted')
print(accuracy, f1).
```

Now, you saw that fine-tuning is an alternative to few-shot classification that allows you to have more control over the model's training. It is generally preferred if you have a larger dataset & need superior performance.

⇒ Multilingual Sentiment Analysis.

Here, you will build a sentiment analysis appⁿ that can classify sentiments in text from multiple languages.

Emotional expression is subjective. It's shaped by cultural background, emotional intelligence, individual personality traits & characteristics & more. While we can communicate emotions clearly in verbal commⁿ due to nuances of tone & inflection, it's harder to determine how a person feels & what emotions they're expressing through words alone.

Fortunately, we can use sentiment analysis to determine the tone - and by extention, the emotion - that text conveys. Here, we are creating an app that uses sentiment analysis to determine what emotions a person is experiencing based on the text input.

The steps for building the Sentiment Analysis app are:

- Step① : Gather Emotion Data
- Step② : Train the Emotion Classifier
- Step③ : Get User Input
- Step④ : Embed the Input
- Step⑤ : Classify Sentiment
- Step⑥ : Display Results
- Step⑦ : Put it all together

Step-by-Step Guide

We'll use the embed-multilingual-v3.0 model via the Cohera API specifically the Embed endpoint — to generate embeddings for user-provided text in real time. The embed-multilingual-v3.0 model generates embeddings for text data in over 100 languages. Then, we'll use these embeddings to train a classifier to predict the emotions the user expressed.

Step① : Gather Emotion Data

We'll use the XED dataset consisting of 8 emotion categories to train the emotion classifier. For this app, the embeddings for this dataset have already been created & stored as xed-with-embeddings.json, along with the labels for each data point.

↳ Step②: Train the Emotion Classifier

The bulk of the code that drives this appn is stored in the sentiment.py file. Let's open it & examine the train-and-sc function line by line.

First, the function reads xed-with-embeddings.Json into a data frame. In this data frame, column df.embeddings contains the embeddings for each sample & column df.labels-text contains the emotion label for each sample. The function transforms the embeddings for each sample into a list, forming data matrix X.

The function uses MultiLabelBinarizer from scikit-learn to one-hot encode df.labels-text & form y, which is a one-hot encoded label matrix. Next, the function executes the train-test split with the test set size configured at 1% of the total set. Afterwards, it instantiates a chain of classifiers with logistic regression as the base classifier.

A classifier chain is a technique for multi-label classification that involves training a chain of binary classifiers, one for each label. The output of each classifier serves as the input to the next classifier in the chain. Consequently, the final output is a vector of binary labels indicating the presence or absence of each emotion.

This demo uses a
logistic regression
& Logistic Regressio
predict the prob.
with chain_model
Subsequently,
on the train set &
calculates the m
In the last line,
determining emotio
A pickle file is
such as trained
a pickle file, u
it from scrat
Then, we save t
into a pickle f
saved model l
in Python.

↳ def train_and_sc
full_df = pd
df = full_df
mlb = Multi
X = np.arr
y = mlb.
classes = m
classes_ma

X_train,